



Society Management & Resident Services Platform with Distributed Backend Architecture

Team ID: 5 | Section: 4 | Members: Challa Sreeja – 2520030219, Bhimavarapu Sai Saranya – 2520030258, Jawaji Advait – 2520030524 | Guide: Dr. Y. Subbarayudu, Asst. Professor
Database Systems Engineering and Distributed Backend Development (25CS1302E) | KLH Bachupally Campus, CSE | PBL 2026-27

Abstract

- A web platform that digitizes residential society operations – billing, visitors, complaints, amenities and notices – for residents, admins and security staff.
- Built on a distributed backend: an API Gateway routes requests to six independent Node.js services, each with its own data store, for scalability and fault isolation.

Problem Statement & Objectives

- Problem:** Societies rely on paper registers, cash/UPI and WhatsApp groups – no payment tracking, visitor audit trail or complaint ticketing.
- Objective 1:** Digitize maintenance billing, payments and receipts.
- Objective 2:** Track visitors, complaints and amenity bookings with live status.
- Objective 3:** Split the backend into independent services behind a secure gateway.
- Scope:** Admin, Resident and Security roles; 6 service modules.

Existing vs Proposed System

Feature	Existing System	Proposed System
Visitors	Paper gate register	Digital log + pre-approval
Maintenance	Cash / informal UPI	Auto bills, payments, receipts
Complaints	Verbal / WhatsApp	Ticket: Pending → Resolved
Notices	Physical notice board	Instant notices + alerts
Gap	Costly, closed apps	Self-hosted, modular, open

Feasibility Study

- Technical:** Pure Node.js (no npm dependencies), REST, SQL; starts with one command – npm run dev.
- Economic:** Fully open-source stack; no licence or vendor subscription; self-hosted or Docker.
- Operational:** Role-based dashboards with demo logins; data reset with npm run reset:data.
- Schedule:** Design, services, SPA and testing completed within the semester timeline.

Results & Testing

- Smoke test checks auth, gateway APIs and frontend end-to-end
- Syntax checks across all services (npm run check)

Conclusion

- Billing, visitors, complaints, amenities and notices unified in one platform
- Meets objectives: role-based access, isolated services, easy deployment

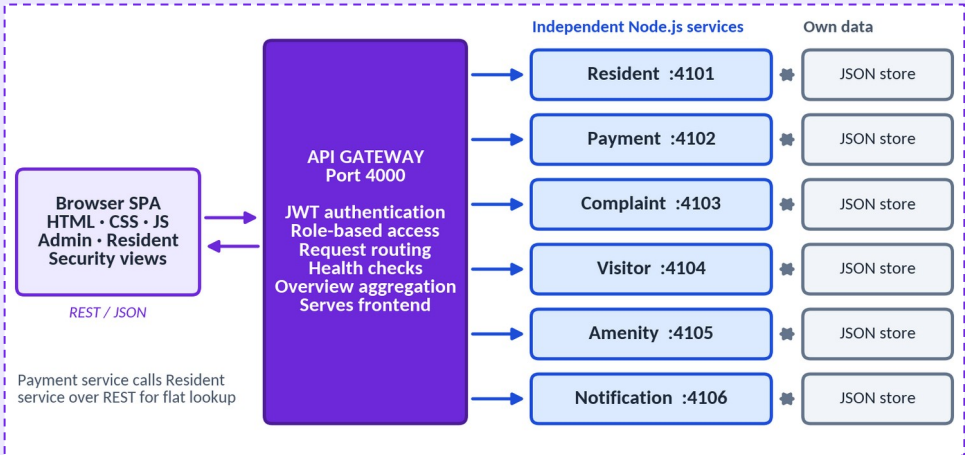
Future Scope

- Resident mobile app and UPI payment gateway integration
- Facial-recognition visitor entry; analytics for committee

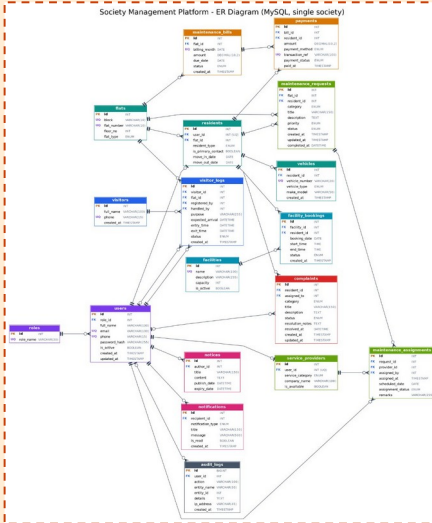
Tools & Technologies

- Front-End:** Single-page app – HTML, CSS, JavaScript
- Back-End:** Node.js (built-in modules), REST APIs, API Gateway
- Database:** Per-service JSON stores + SQL schema (PostgreSQL) with views, trigger and stored procedure
- Security:** JWT authentication, role-based access control
- Tools:** Git & GitHub, Docker Compose, Postman, VS Code

System Architecture



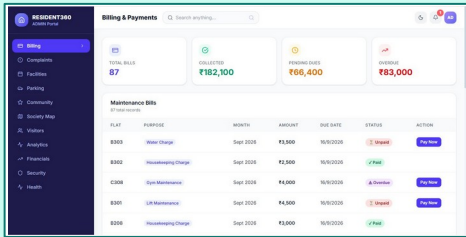
ER Diagram & Database Design



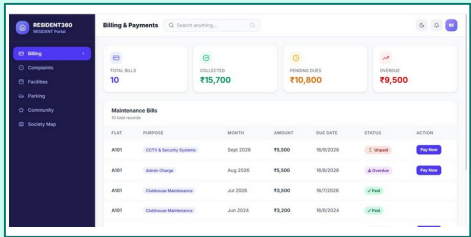
18 tables, grouped by module

- Access:** roles, users
- Residents:** flats, residents, vehicles
- Billing:** bills, payments
- Gate:** visitors, visitor logs
- Services:** requests, assignments, providers
- Community:** complaints, facilities, bookings, notices, notifications
- Audit:** audit logs

Implementation Screenshots



Admin portal



Resident portal

